AIWareK: Compiling PyTorch Model for AI Processor Using MLIR Framework

Hyunjeong Kwon
AI SoC Research Division
ETRI
Daejeon, Republic of Korea
kwonhj@etri.re.kr

Hyun Mi Kim
AI SoC Research Division
ETRI
Daejeon, Republic of Korea
chaos0218@etri.re.kr

Chun-Gi Lyuh
AI SoC Research Division
ETRI
Daejeon, Republic of Korea
cglyuh@etri.re.kr

Jin-Kyu Kim
AI SoC Research Division
ETRI
Daejeon, Republic of Korea
kimjk@etri.re.kr

Jinho Han
AI SoC Research Division
ETRI
Daejeon, Republic of Korea
soc@etri.re.kr

Youngsu Kwon
AI SoC Research Division
ETRI
Daejeon, Republic of Korea
yskwon@etri.re.kr

2022 IEEE 4th International Conference on Artificial Intelligence Circuits and Systems (AICAS) | 978-1-6654-0996-4/22/$31.00 ©2022 IEEE | DOI: 10.1109/AICAS54282.2022.9869913

***Abstract*—Deep learning compiler becomes necessary with the active research on AI hardware. This work compiles PyTorch models into target hardware codes using MLIR framework. The compiler first constructs a graph from the PyTorch model using TorchScript tracing. To construct the graph, our domain-specific parser generates an abstract syntax tree using tokens generated from the lexer. Then, the graph IR (GIR) is built and lowered into the kernel IR (KIR) and the processor IR (PIR) in MLIR framework. PIR becomes the input of the backed compiler that generates target machine codes. Experimental result shows that AIWareK compiled ResNet18 in 7.67 seconds, yielding 1.16e-03 mean absolute error.**

***Index Terms*—MLIR, compiler, deep learning, AI hardware**

I. Introduction

Deep learning compiler processes deep learning models to generate target hardware codes [1]. The compiler parses the input model and executes passes including the lowering of intermediate representation (IR) and the optimization on IR [2]. Lowering pass is the translation of one IR to another IR, which allows to apply different optimization passes on different IR levels. Optimization pass is the simplification and regularization processes on an IR.

Multi-level intermediate representation framework (MLIR) provides the extensible compiler infrastructure [3]. Previous research has widely used MLIR to implement the deep learning compiler on various applications [4]–[9]. [4] supports TensorFlow framework as the input model, and allows CPU, GPU, and TPU as the target hardware. Other works [5], [6] support various machine learning frameworks such as PyTorch and ONNX, but have limitations on the target hardware due to the lack of backend compilers.

This work implements a novel deep learning compiler, called AIWareK. AIWareK constructs an abstract syntax tree (AST) from the PyTorch model and lowers IRs to generate target hardware codes (Fig. 1). Currently we support CPU and an AI hardware, AB9 [10], as the target hardware. Our main contributions are:

- We design the three levels of IRs, GIR, KIR, and PIR, and the two novel dialects, torch and AB9. We also propose the corresponding translation passes for lowering between the IRs.
- We implement the novel lexer and parser to construct an AST from the input PyTorch model. TorchScript is used to figure out the graph structure of the deep learning model.
- We propose a method to divide the input model into several sub-graphs according to the target hardware in Python.
- We run the input PyTorch model on AB9 using the target machine codes generated from the MLIR-based deep learning compiler, AIWareK. AB9 is a systolic array-based accelerator, and allows to accelerate the runtime of the inference.

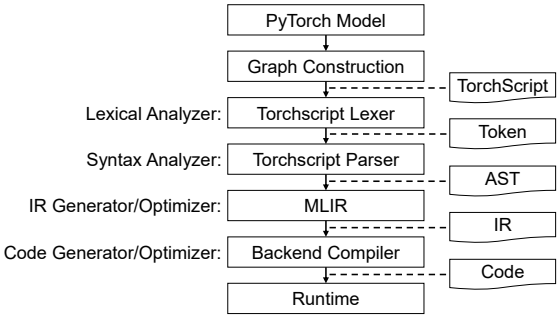


Fig. 1. Flowchart of AIWareK

II. AIWareK: PyTorch Model Compiler using MLIR

In this section, we look into the steps of AIWareK (Fig. 1) in detail with priority given to graph construction (II-A), lexer

This research was supported by the Institute of Information Communications Technology Planning Evaluation (IITP) grant funded by the Korean government (MSIT) (No. 2018-0-00195, Artificial Intelligence Processor Research Laboratory).

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on September 10,2024 at 07:54:56 UTC from IEEE Xplore. Restrictions apply.

```
graph(%self.1 : __torch__.torch.nn.modules.container.Sequential,
      %input : Float(1, 1, 10, 10, strides=[100, 100, 10, 1], requires_grad=1, device=cpu)):
  %_0 : __torch__.torch.nn.modules.conv.Conv2d = prim::GetAttr[name="_0"](%self.1)
  %4 : bool = prim::Constant[value=1](), scope: __module._0
  %5 : int = prim::Constant[value=0](), scope: __module._0
  %6 : bool = prim::Constant[value=0](), scope: __module._0
  %7 : int = prim::Constant[value=1](), scope: __module._0
  %8 : int = prim::Constant[value=3](), scope: __module._0
  %9 : int = prim::Constant[value=2](), scope: __module._0
  %bias : Tensor = prim::GetAttr[name="bias"](%_0)
  %weight : Tensor = prim::GetAttr[name="weight"](%_0)
  %12 : int[] = prim::ListConstruct(%9, %9), scope: __module._0
  %13 : int[] = prim::ListConstruct(%8, %8), scope: __module._0
  %14 : int[] = prim::ListConstruct(%7, %7), scope: __module._0
  %15 : int[] = prim::ListConstruct(%5, %5), scope: __module._0
  %16 : Float(1, 1, 7, 7, strides=[49, 49, 7, 1], requires_grad=1, device=cpu) =
        aten::_convolution(%input, %weight, %bias, %12, %13, %14, %6, %15, %7, %6, %
        6, %4, %4), scope: __module._0
  return (%16)
attr(
  _0.weight
  tensor([[[[ 0.3040, -0.2644, -0.2027],
            [-0.3116, -0.1439, 0.1628],
            [-0.1099, 0.1828, 0.1151]]]])
  _0.bias
  tensor([0.1857])
)
```

Fig. 2. Graph Construction Using TorchScript

TABLE I
15 TYPES OF TOKENS FOR LEXER

tok_semicolon	tok_eof	tok_return	tok_var
tok_parenthese_open	tok_bracket_open	tok_sbracket_open	tok_attr
tok_parenthese_close	tok_bracket_close	tok_sbracket_close	tok_torch
tok_graph	tok_number	tok_identifier	

and parser (II-B), and the frontend compiler (II-C). LLVM [11] and AIWareRT [12] are adopted as the backend compilers.

A. Graph Construction

AIWareK extracts a graph data structure from the input PyTorch model, *torch.nn.Module*, that consists of *torch.nn* operators. TorchScript contains the graph information of a PyTorch model, and the script module is extracted using *jit.trace* function. The member variable, *inlined_graph*, of the script module returns the text-based graph information of the model (Fig. 2). The member function, *state_dict()*, of a *torch.nn.Module* returns the list of attributes of all operators in the model. The attribute keys and values are also extracted as shown in the last 8 lines in Fig. 2.

Depending on the target hardware for each operator in the model, a subgraph that ought to pass AIWareK is replaced with a custom model. The custom model is also a PyTorch model, and the forward method of the custom model runs AIWareK (Fig. 1).

B. Lexer and Parser

The text-based graph data structure is parsed to construct the abstract syntax tree (AST), which is the input of building GIR in MLIR. Lexer extracts tokens from the input in Fig. 2, and classifies the types of tokens according to the predetermined 15 types (Table I). Parser is a syntax analyzer that builds an AST according to the types of tokens, and the AST has 12 types of nodes (Table II).

TABLE II
TYPES OF AST NODES

ModuleAST	TorchOpExprAST	ListExprAST	VarDeclExprAST
FunctionAST	TensorExprAST	LiteralExprAST	ReturnExprAST
PrototypeAST	NumberExprAST	VariableExprAST	CallExprAST

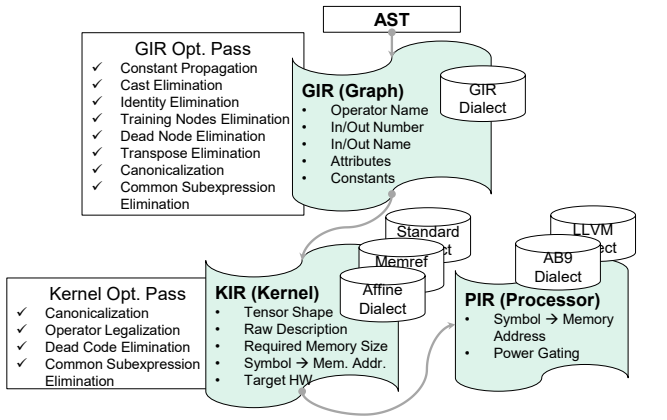


Fig. 3. Lowering and Optimization Flow of MLIR with GIR, KIR, and PIR

For example, for the forth line in Fig. 2, the lexer delivers the tokens, *%4*, *bool*, *prim::Constant*, *value*, and *1* to the parser. Then the parser makes a node, *VarDeclExprAST*, and saves the name (*%4*), the type (*bool*), and the value (*1*) to the node. The parser also adds the node to the body that represents the input graph, *FunctionAST*.

C. MLIR: Frontend Compiler

MLIR framework is used to implement the frontend compiler of AIWareK. In this step, *FunctionAST* is transformed into GIR in MLIR, and the GIR is optimized and lowered into other IRs. We propose three different levels of IRs and two dialects, torch and AB9 dialects. We also use four existing dialects, affine, memref, standard, and LLVM dialects, that are provided by MLIR [3]. Finally, in GIR, all the information on the model is represented using operators defined in GIR dialects. In the lowering pass from GIR to KIR, the operators are transformed into another set of operators that are defined in one of affine, memref, and standard dialects. In PIR, only AB9 and LLVm dialects are allowed according to the target HWs.

MLIR also includes optimization passes as well as the lowering passes. As in Fig. 3, AIWareK applies optimization passes such as node elimination and common subexpression elimination on IRs. The lowering pass from GIR to KIR includes the shape inference and the description of tensor calculation using three dialects in KIR. In this pass, an operator-level representation is unrolled into the actual calculation-based representation including arithmetic operations, for loop, or memory load and store operations. KIR to PIR lowering consists of lowering to LLVM dialect and lowering to AB9 dialect.

Finally, PIR is transferred to the backend compiler that generates the target machine code. PIR with the LLVM dialect

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on September 10,2024 at 07:54:56 UTC from IEEE Xplore. Restrictions apply.

TABLE III
COMPILE/INFERENCE TIME AND ABSOLUTE ERROR FOR RUNNING RESNET18 ON AB9

Torch to AST [s]	28.9
Compilation Time [s]	7.67
Inference Time [s]	5.36×10^{-2}
Mean Absolute Error	1.16×10^{-3}

is processed using LLVM backend with a target hardware, CPU. AIWareRT [12] is used as the backend compiler to process PIR with the AB9 dialect, and the generated codes are run on the deep learning accelerator, AB9.

III. EXPERIMENTS

We validate AIWareK using ResNet18 [13] and run the model on AB9. ResNet18 contains 69 operators with 8 different types. There are 11,689,512 trainable parameters in the model. AIWareK is implemented on Intel(R) Xeon(R) Gold 5222 CPU @ 3.80GHz.

Torch to AST in Table III represents the runtime consumed by lexical and syntax analyzer, which is performed only once when the input model is determined. *Compilation time* and *inference time* show that AIWareK compiles ResNet18 in 7.67 seconds and AB9 processes the machine codes in 5.36e-02. Compared to the inference time when running it on CPU, 9.13e-02 seconds, AB9 reduced the time by over 41%. The last line of Table III indicates the mean absolute error over all elements of the output vector.

IV. CONCLUSION

AIWareK directly compiles the PyTorch model using MLIR framework, and generates target machine codes. In this work, we implemented the domain-specific lexer and parser, and proposed novel IRs and dialects to compile PyTorch models efficiently. Experimental result shows that AIWareK compiled ResNet18 in 7.67 seconds yielding 1.16e-03 mean absolute error.

REFERENCES

[1] M. Li *et al.*, "The deep learning compiler: A comprehensive survey," *IEEE Transactions on Parallel and Distributed Systems*, vol. 32, no. 3, pp. 708–727, 2020.

[2] A. V. Aho *et al.*, *Compilers: Principles, Techniques, and Tools (2nd Edition)*. Addison Wesley, August 2006.

[3] C. Lattner *et al.*, "Mlir: Scaling compiler infrastructure for domain specific computation," in *2021 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, pp. 2–14, IEEE, 2021.

[4] M. Abadi *et al.*, "TensorFlow, Large-scale machine learning on heterogeneous systems." https://https://github.com/tensorflow/tensorflow, 2015.

[5] "torch-mlir." https://github.com/llvm/torch-mlir.

[6] T. Jin *et al.*, "Compiling onnx neural network models using mlir," *arXiv preprint arXiv:2008.08272*, 2020.

[7] T. Gysi *et al.*, "Domain-specific multi-level ir rewriting for gpu: The open earth compiler for gpu-accelerated climate simulation," *ACM Transactions on Architecture and Code Optimization (TACO)*, vol. 18, no. 4, pp. 1–23, 2021.

[8] U. Bondhugula, "High performance code generation in mlir: An early case study with gemm," *arXiv preprint arXiv:2003.00532*, 2020.

[9] A. McCaskey and T. Nguyen, "A mlir dialect for quantum assembly languages," *arXiv preprint arXiv:2101.11365*, 2021.

[10] Y. C. P. Cho *et al.*, "Ab9: A neural processor for inference acceleration," *ETRI Journal*, vol. 42, no. 4, pp. 491–504, 2020.

[11] C. Lattner and V. Adve, "Llvm: A compilation framework for lifelong program analysis & transformation," in *International Symposium on Code Generation and Optimization, 2004. CGO 2004.*, pp. 75–86, IEEE, 2004.

[12] H. Kim *et al.*, "Automated optimization for memory-efficient high-performance deep neural network accelerators," *ETRI Journal*, vol. 42, no. 4, pp. 505–517, 2020.

[13] K. He *et al.*, "Deep residual learning for image recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 770–778, 2016.

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on September 10,2024 at 07:54:56 UTC from IEEE Xplore. Restrictions apply.